# Ansible: from one host to a fleet

A working model for an SRE directing and reviewing agent-written automation

Prepared for Jeff

20 October 2026

# Contents

**Reading on the plane** **4**

**1 Predict the run** **5**
1.1 A remote programme, with a state-aware vocabulary . . 5
1.2 Predict ordering per host . . . . . 6
1.3 Where judgement belongs . . . . . 7

**2 Prove the target before changing it** **8**
2.1 Identity is part of configuration . . . . . 8
2.2 Selection has two stages . . . . . 9
2.3 Access, privilege and the controller's configuration . . . . 10

**3 Configure one real service** **11**
3.1 Define the contract first . . . . . 11
3.2 Read the tasks as a causal chain . . . . . 12
3.3 The complete template and handler . . . . . 13
3.4 The play controls selection and verification . . . . . 15

**4 Trace data before adding abstraction** **17**
4.1 Variables are inputs, not a Python execution frame . . . 17
4.2 Learn the precedence relationships you need . . . . . 19
4.3 Grow structure in response to pressure . . . . . 20

**5 Build evidence in layers** **21**
5.1 A preview answers a narrower question than a run . . . . 21
5.2 Try the complete remote example later . . . . . 22
5.3 A local experiment in convergence . . . . . 23

**6 Control a fleet change** **25**
6.1 Concurrency is not an availability policy . . . . . 25
6.2 Decide what failure should stop . . . . . 26
6.3 Turn a batch into a service-aware procedure . . . . . 26

**7 Diagnose partial completion and recover deliberately** **28**
7.1 Reconstruct the last proven state . . . . . 28
7.2 The lost-notification trap . . . . . 29
7.3 Recovery is a new, bounded change . . . . . 30

**8 Keep operations understandable** **32**
8.1 Treat secrets as a data path . . . . . 32
8.2 Pin the executable environment, then plan upgrades . . . 33
8.3 Read an inherited repository before refactoring it . . . . 34

**9 Share ownership across tools** **36**
9.1 Establish a single writer for each resource . . . . . 36
9.2 Containers move some of the boundary . . . . . 37

**10 Worked answers** **39**
10.1 1. Earlier effects survive . . . . . 39
10.2 2. A limit must resolve to the intended inventory identity 39
10.3 3. Report and control the right transition . . . . . 39
10.4 4. Extra variables win here; ordinary CLI flags need care 40
10.5 5. Convergence and a 200 response leave important gaps 40
10.6 6. A batch can change several hosts before its gate fails . 41
10.7 7. Separate missed activation from rejected activation . . 41
10.8 8. Ciphertext in Git does not protect every plaintext copy 42
10.9 9. Give the file and lifecycle one owner each . . . . . 42

**11 A field reference for later** **43**

**12 Sources, artifacts and verification** **44**

# Reading on the plane

This tutorial is about understanding what Ansible will do well enough to entrust it with real machines. You already know Linux operations and Python. We will build on that experience: processes, files, SSH, exit status and failure domains are familiar; Ansible adds its own execution and data model.

Your use case spans personal projects and a Jellybooks fleet of roughly 30–50 hosts. Some services run outside Docker. Terraform is on your near-term roadmap, and a later move towards a pod is possible. We will give each tool a clear boundary without assuming what that future platform will be.

Allow two to three hours, including the short prediction and review exercises. Read chapters 1–7 in order. Chapter 8 helps when adopting an inherited repository; chapter 9 is an optional bridge to Terraform and containers. Worked answers are collected near the end. You need neither a terminal nor the linked documentation to follow the argument. Commands are there to make behaviour precise and to try later. Pause at each exercise before reading its answer.

The target is **ansible-core 2.21.5**, with Python 3.12 on the controller. Concrete remote examples assume **dedicated Ubuntu 24.04 VMs with systemd**, Python 3 and SSH. Your actual host distributions were not specified; Ubuntu is a teaching assumption. Package names, service managers and paths need checking elsewhere. Core 2.21 supports controller Python 3.12–3.14 and target Python 3.9–3.14. The separate community `ansible` package bundles additional collections; its version number is not the Core version. This tutorial needs Core only. Release support, pinned release.

The remote example replaces the **whole nginx configuration** on disposable status-service hosts. It is a small, deliberately bounded non-Docker service, not a recommended Jellybooks deployment. Never point it at an existing web host. Its loopback-only endpoint avoids requiring a public listener or firewall change.

The editable source is `tutorial.md`. The `examples/` directory contains complete files. A build guide and recorded verification are in `README.md` and `verification.md`. These filenames identify supporting artifacts; all essential explanations and remote configuration appear in this PDF.

# 1 Predict the run

## 1.1 A remote programme, with a state-aware vocabulary

Imagine a Python programme that iterates over machines, connects over SSH, inspects a file or package, and changes it if necessary. Ansible provides the machinery for that pattern: selecting targets, obtaining connection details, transferring and executing modules, evaluating expressions, handling results and reporting progress. A **module** is the implementation of an operation such as installing a package or rendering a file. A **task** invokes a module with arguments and execution policy. A **play** maps a set of hosts to an ordered sequence of work. A **playbook** is a YAML list of plays.

The **control node** runs Ansible and reads your repository and credentials. The **managed nodes** are its targets. Ordinary Linux management uses SSH and usually Python on the target; it does not require a persistent Ansible daemon there. Modules normally perform their host operations on the target. Templates are rendered on the controller, and some actions, including debugging, have controller-side behaviour. A task delegated elsewhere is another explicit exception. Ask where each action runs whenever the answer affects access, credentials or network visibility. Concepts, templating.

This task is a specification of file content, not an instruction to append text:

```yaml
- name: Keep the maintenance notice correct
  ansible.builtin.copy:
    content: "Managed by the infrastructure repository.\n"
    dest: /etc/motd
    owner: root
    group: root
    mode: '0644'
```

It assumes a privileged task context. `copy` is from the `ansible.builtin` collection included in Core. Fully qualified names make the implementation unambiguous. The target path is on the managed node. The content is supplied by the controller. The indentation assigns the module arguments; `name` is a human label, not a function identifier or

an acceptance test.

Compare that with `echo ... >> /etc/motd`. Appending repeats a side effect on each run. A state-aware file task can compare desired and actual content, install the specified bytes and permissions, and report whether it changed anything. **Idempotence** means repeat execution leaves the desired state unchanged. It is a property of the operation and its arguments, not a property granted by putting a command inside YAML. A task that appends a line unconditionally remains non-idempotent. A package task requesting `latest` may change next week because its desired result depends on the repository at run time. Playbook execution and idempotence.

Think of Ansible as an ordered series of local convergence operations. It has neither a general transaction spanning those operations nor a persistent process continuously reconciling them. If someone edits the file tomorrow, it stays edited until another run repairs it, unless you arrange a separate schedule. A successful run is a dated observation and a sequence of effects.

## 1.2 Predict ordering per host

Suppose a play selects three hosts and contains package, configuration and service tasks. With the default **linear strategy**, Ansible completes the package task across participating hosts before proceeding to the configuration task. Work within a task can happen concurrently. The default five **forks** bound worker parallelism; they do not define a safe disruption budget. A failure normally removes the affected host from subsequent ordinary work while healthy hosts continue. There is no automatic undo of earlier changes. Strategies, errors.

For operational reasoning, keep three kinds of state separate:

| State | Example | Evidence |
|---|---|---|
| Desired | Repository says release `r2` | Reviewed revision and effective variables |
| Installed | Configuration on disk says `r2` | Target file inspection and validation |
| Live | Process still serves `r1` | Actual response, process state and logs |

Ansible does not collapse these into one truth. The file task may have succeeded while activation failed. A service manager can report a process running while that process serves the wrong data. This separation will explain most of the interesting failures later.

The result vocabulary is also evidence with a limited meaning. `ok` means the task succeeded without reporting a change. `changed` means it reported a change. `skipped` means it was not performed for that host. `failed` and `unreachable` distinguish an operation failure from inability to reach the host. A task can customise its changed and failure conditions; therefore a green recap is only as good as those conditions. Always read the task implementation behind a surprising success.

## 1.3 Where judgement belongs

An agent can usually write straightforward module calls, scaffold a role and fix mechanical YAML mistakes. You retain decisions about ownership, target selection, privilege, acceptable disruption and evidence of correctness. You also decide whether an operation can safely be repeated. That decision cannot be inferred from a task named "Ensure database migration is applied". It needs the migration system's own semantics.

**Ask an agent:** "Trace this play on three hosts if the configuration task fails on the second host." A good answer gives the selected hosts, strategy, preceding side effects, tasks still eligible on each host and handler implications. A weak answer says "Ansible is declarative, so rerunning will fix it" without explaining intermediate state.

**Exercise 1 — predict.** A play writes a valid configuration, starts a service and then fails its health check. Which earlier effects remain? What would you need to inspect before deciding to rerun? Write a three-sentence answer.

# 2 Prove the target before changing it

## 2.1 Identity is part of configuration

An **inventory** describes host identities, groups and connection data. Start with a small explicit file whose output you can inspect. Our example is `examples/inventories/lab/hosts.yml`:

```
all:
  children:
    status:
      hosts:
        lab-status-01:
          ansible_host: 192.0.2.11
        lab-status-02:
          ansible_host: 192.0.2.12
```

The addresses are documentation placeholders. Replace them with two dedicated lab VMs before running anything remotely. `lab-status-01` is the inventory identity, available as `inventory_hostname`. `ansible_host` is the address used to connect. A hostname pattern normally selects the identity, not an arbitrary address you remember belongs to it. Aliases also allow two names to describe the same physical target accidentally. A correct host count alone cannot detect that duplication. Inventory, patterns.

Adjacent `group_vars/all.yml` supplies connection defaults:

```
ansible_user: deploy
ansible_python_interpreter: /usr/bin/python3
```

`group_vars/status.yml` holds this environment's service data:

```
status_port: 8080
status_release: r1
```

Groups are sets with attached variables, not classes with a tidy single-parent inheritance hierarchy. A host can be both `web` and `staging`, or both `database` and `production`. Do not assume those group names are mutually exclusive. Keep variables beside the inventory whose environment they describe; overlapping definitions need deliberate precedence, discussed in chapter 4.

These commands, run from `examples/`, inspect the chosen inventory:

```
ansible-inventory -i inventories/lab/hosts.yml --graph
ansible-inventory -i inventories/lab/hosts.yml \
  --host lab-status-01
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --list-hosts --limit lab-status-01
```

The graph should contain two members of `status`; the final command should select one. `--host` displays merged inventory variables, not every value a later task might acquire from roles, registered results or play variables. Its output may contain secrets in a real repository: inspect locally and avoid pasting it into a public ticket.

## 2.2 Selection has two stages

A play's `hosts: status` selects candidates. `--limit lab-status-01` narrows that set. It does not turn a play for databases into a play for web servers. With appropriate groups, the quoted pattern `'web:&staging'` selects their intersection; `'web:!database'` excludes database members. Quote punctuation-rich patterns so your shell does not interpret them. Preview the exact invocation, including inventory, limit and extra variables, before executing it. Pattern semantics.

Avoid replacing the inventory with `-i '192.0.2.11,'` just to get a quick command working. Inline inventory creates a different inventory source and can lose the groups and variables that made the intended run meaningful. Likewise, a typo that selects zero hosts can leave a command doing no useful work. Acceptance should require the expected identities and count, not merely a zero exit status.

Separate environment inventories are a useful recommendation at your scale. For example, `inventories/personal`, `inventories/staging` and `inventories/production` make selection explicit. They are organisational boundaries rather than access controls: shared credentials and a mistaken `-i` can still reach production. Pair them with appropriate credentials and a reviewable operational command. Do not combine all inventories for convenience and assume `--limit` will always save you.

## 2.3 Access, privilege and the controller's configuration

SSH login and **become** are separate. `ansible_user: deploy` chooses the remote login. `become: true` asks for privilege escalation, normally using sudo; `become_user` specifies the target account. Logging in successfully does not prove sudo works. `--ask-become-pass` supplies a prompt when required; a remote SSH password uses a different option. Our remote play uses root privilege to install packages and manage `/etc`, while the local exercises need none. Privilege escalation.

The lab assumes you already have verified SSH host keys and an authorised account. `ansible.builtin.ping` tests Ansible connectivity and a usable Python runtime; it is not ICMP ping or proof the application works. After inspecting targets, a bounded connection check is:

```
ansible -i inventories/lab/hosts.yml lab-status-01 \
  -m ansible.builtin.ping
```

Bootstrap is a separate dependency. A new machine needs an address, trusted host identity, SSH access, suitable Python and the required sudo policy before ordinary Linux tasks can run. A cloud image or cloud-init can establish that minimal contract. The `raw` module can help bootstrap Python because it does not require target Python, but bootstrap still needs transport and privilege. Avoid turning off host-key checking across the fleet to work around missing trust. Ping, raw.

The controller has configuration too. `ansible --version` reports the Core version and selected configuration file. `ansible-config dump --only-changed` helps find deviations from defaults. Ansible searches configuration locations in a defined order, using the first applicable file; a world-writable current directory is a documented special case. Environment variables can affect behaviour. Record these details when two operators see different results. Configuration.

**Ask an agent:** "Show that this command affects exactly the two intended staging hosts, with the intended SSH identity." Good evidence includes the full command, selected config, inventory output, actual aliases and effective connection data. "The play says `hosts: web`" is inadequate because the inventory determines what `web` contains.

**Exercise 2 — review.** An agent reports success after running a play with `--limit 192.0.2.11`, while the inventory calls that host `lab-status-01`. What must you check? Why is changing to an inline address inventory a poor automatic fix?

# 3 Configure one real service

## 3.1 Define the contract first

Our service returns an exact, host-specific line on a loopback HTTP endpoint:

```
ok lab-status-01 r1
```

The final newline is part of the expected content. The contract is: nginx is installed; the complete lab configuration is validated and owned by this role; the service is running and enabled at boot; `/healthz` on port 8080 returns the expected identity and release. Other paths return 404. No TLS, public routing or load-balancer registration is implied. A host-local check verifies the local process path, not an external client's route to the service.

The small role has conventional directories:

```
roles/status_service/
  defaults/main.yml
  tasks/main.yml
  handlers/main.yml
  templates/nginx.conf.j2
```

A **role** packages related tasks, handlers, templates and defaults. It is a packaging and reuse boundary, not a scheduler or separately running component. Roles do not automatically isolate variables and handlers: variables can enter play scope, and handlers share the play's scope. Use distinctive input and handler names when composing roles. Read its files as a small programme. Here, putting the files in a role also makes the later growth discussion concrete; one play with tasks would be reasonable for a single small personal host.

## 3.2 Read the tasks as a causal chain

The following is the complete `roles/status_service/tasks/main.yml`:

```yaml
---
- name: Reject unsafe template inputs
  ansible.builtin.assert:
    that:
      - status_port is integer
      - status_port >= 1024 and status_port <= 65535
      - status_release is string
      - status_release is match('^[A-Za-z0-9_-]+$')
      - inventory_hostname is match('^[A-Za-z0-9_.-]+$')

- name: Install nginx without package-triggered service startup
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
    cache_valid_time: 3600
    policy_rc_d: 101

- name: Render and validate the complete lab nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
    validate: /usr/sbin/nginx -t -c %s
  notify: Reload status nginx

- name: Keep nginx running and enabled at boot
  ansible.builtin.systemd_service:
    name: nginx
    state: started
    enabled: true
```

Input assertions have a practical purpose. A release label is inserted into an nginx quoted string; allowing arbitrary quotes, newlines or directives would make data become configuration syntax. Restricting

this teaching fixture to simple labels avoids that ambiguity. An integer assertion also catches the common mistake of passing a port as a string. For ad hoc numeric overrides use JSON, for example `-e '{"status_port": 8081}'`, rather than assuming every command-line representation preserves types.

`apt` manages the package, including its metadata cache policy. `present` does not request an upgrade when an adequate installed package already satisfies that state. It also does not pin a specific nginx build. A new VM can receive a newer repository version than an old one. Decide separately whether your real fleet uses package version constraints, a snapshot repository, or an explicit patching workflow. Cache refreshes are observable maintenance effects; a fresh second run can be quiet while a later refresh legitimately reports change.

Debian-family package scripts can start services during installation, before your following task writes configuration. `policy_rc_d: 101` suppresses that package-triggered startup for this operation. It does not stop an already running service or implement a fleet-wide deployment policy. The module's target requirements include `python3-apt`; current versions can arrange dependency installation. Our example assumes working apt repositories and target egress. Apt module.

The template module renders a candidate, validates it on the target, then installs it. `%s` is replaced by the temporary candidate filename. The command does not run through a shell, so a pipe or `&&` is not a validation programme. This command needs a **complete nginx configuration**: a `server` fragment on its own would not be a valid main configuration. Quoting `'0644'` preserves the intended mode. Template validation, nginx switches.

A rejected candidate should leave the previous destination configuration in place. That is a file-level property, not a transaction covering package installation, service activation and future tasks. If nginx was just installed, that installation remains even when template validation fails.

## 3.3 The complete template and handler

`roles/status_service/templates/nginx.conf.j2` contains:

```
# This role owns the complete configuration on dedicated lab
 ↪  hosts.
user www-data;
worker_processes auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log warn;

events {
    worker_connections 128;
}

http {
    access_log /var/log/nginx/access.log;
    server {
        listen 127.0.0.1:{{ status_port }};
        server_name _;
        location = /healthz {
            default_type text/plain;
            return 200 "ok {{ inventory_hostname }} {{
 ↪  status_release }}\n";
        }
        location / {
            return 404;
        }
    }
}
```

We intentionally own the whole file and do not include the distribution's `sites-enabled` directory. This makes the validation unit obvious and removes unrelated default sites. On an inherited web host that would remove existing behaviour: a precise demonstration of why ownership must be agreed before a plausible-looking role is applied.

A **handler** is a task scheduled by a changed task's `notify`. The file `roles/status_service/handlers/main.yml` contains:

```
---
- name: Reload status nginx
  ansible.builtin.systemd_service:
    name: nginx
    state: reloaded
```

A notification means "this host needs this action", not "call this function immediately". Repeated notifications before execution deduplicate, and handlers run in their definition order rather than notification order. Our play explicitly flushes pending handlers before its behavioural check. A flush applies to pending handlers in the play, so place it with awareness of other roles as the play grows. Handlers.

`state: started` is an idempotent request to have the service running. `state: reloaded` requests a reload each time that task runs and starts a stopped service. The handler makes that unconditional operation conditional on a relevant change. `enabled: true` governs boot behaviour independently. A systemd `daemon_reload` rereads unit definitions; it does not make nginx reread `nginx.conf`. Those are different layers of configuration. Systemd module.

The role's `defaults/main.yml` provides overridable values:

```yaml
---
status_port: 8080
status_release: r1
```

These match our lab inventory. The duplication is deliberate: role defaults express the reusable contract; inventory records the environment's chosen values. For a very small repository you could omit explicit inventory values until they differ, but make sure a reviewer can find where a real environment's policy lives.

## 3.4 The play controls selection and verification

Here is the complete `site.yml`:

```yaml
---
- name: Configure dedicated lab status hosts
  hosts: status
  become: true
  gather_facts: true
  serial: 1
  any_errors_fatal: true

  pre_tasks:
    - name: Require the documented disposable Ubuntu environment
      ansible.builtin.assert:
```

```
      that:
        - ansible_facts['distribution'] == 'Ubuntu'
        - ansible_facts['distribution_version'] == '24.04'
        - ansible_facts['service_mgr'] == 'systemd'
      fail_msg: Use dedicated Ubuntu 24.04 systemd lab VMs.

- roles:
    - status_service

- tasks:
    - name: Apply pending service changes before checking
↪     behaviour
      ansible.builtin.meta: flush_handlers

    - name: Check the response served by this host
      ansible.builtin.uri:
        url: "http://127.0.0.1:{{ status_port }}/healthz"
        return_content: true
        status_code: 200
      register: status_health
      until: >-
        status_health.status | default(0) == 200 and
        status_health.content | default('') ==
        'ok ' ~ inventory_hostname ~ ' ' ~ status_release ~ '\n'
      retries: 5
      delay: 2
      when: not ansible_check_mode
```

`gather_facts` obtains target data; the pre-task rejects an unsupported environment before installing packages. It does not prove a machine is disposable. Inventory and operator selection must establish that. `serial` and `any_errors_fatal` form a conservative teaching rollout; their exact semantics are in chapter 6.

`register` captures this host's result in a dictionary. `until` retries until both HTTP status and body match. Jinja's `~` concatenates string representations; `default` allows the expression to handle connection errors with missing fields. The health task normally runs on the target, so `127.0.0.1` is that host's loopback. The query has no supported predictive check mode; we explicitly skip it in check runs. It cannot tell us how a not-yet-applied service would behave. URI module, retrying

tasks.

On a fresh host the package and configuration change, the service starts, the handler reloads it, and the response must match. On an immediate second run with unchanged inputs, expect no configuration change and no reload notification; health is still checked. If the release becomes `r2`, the file changes, reload runs and the expected response changes. That is the causal path your review should establish.

**Ask an agent:** "Show that a rejected configuration cannot replace the live file, and that a successful run actually serves the requested release." A good answer distinguishes candidate validation, disk installation, reload, host-local response and any external path still untested. "The handler succeeded" leaves those distinctions unresolved.

**Exercise 3 — review.** An agent proposes replacing the handler with `state: restarted` in an ordinary task on every run, and adding `daem on_reload: true` "so nginx sees its config". Identify the two mistaken assumptions and describe the useful evidence for deciding between reload and restart.

# 4 Trace data before adding abstraction

## 4.1 Variables are inputs, not a Python execution frame

For a Python programmer, YAML is data describing operations and policy. Jinja expressions compute arguments from the current host's variables. Do not imagine every variable is assigned once when the file is parsed, or that a global assignment necessarily affects all hosts. Effective values depend on scope, precedence and when an expression is evaluated.

Ansible **facts** describe observed host properties, usually gathered by the `setup` module at the beginning of a play. `ansible_facts['distr ibution']` is an observation, whereas `status_release` is our desired input. `inventory_hostname` is inventory identity, whereas a hostname fact comes from the target. Cached facts, if enabled, are another dated observation. Do not use a potentially stale fact as proof of a current service's health. Facts and registered variables.

A `register` result belongs to the host whose task ran. It can contain `rc`, `stdout` or module-specific fields, not just a boolean. A skipped task can still create a result marked as skipped, without the fields a successful execution would have supplied. Conditions must account for that shape. When a task loops, registered results usually describe multiple per-item outcomes rather than one simple `stdout`.

The following diagnostic excerpt belongs inside a play on a systemd host:

```
- name: Observe nginx without changing it
  ansible.builtin.command:
    argv: [systemctl, is-active, nginx]
  register: nginx_active
  changed_when: false
  failed_when: nginx_active.rc not in [0, 3]

- name: Explain an inactive service
  ansible.builtin.debug:
    msg: "nginx is inactive; inspect its journal before
 ↪  restarting"
  when: nginx_active.rc == 3
```

We deliberately accept systemctl's expected active/inactive outcomes; other errors remain errors. This is an observation, not a desired-state task. `changed_when: false` accurately labels a read-only query. On a mutating command it would merely hide the reported change and could suppress a necessary handler. Conditions such as `when` are already Jinja expressions: write a boolean expression directly, without another pair of `{{ ... }}`. Current Core is strict about conditional types; strings such as `"false"` need deliberate conversion rather than reliance on Python-like truthiness. Conditionals, 2.19 templating changes.

Use `command` with `argv` when an external programme is necessary and a module cannot express the operation. `command` does not run through a shell. It cannot interpret `|`, redirection or `&&` as shell syntax. `shell` can, but then quoting, injection, exit status and repeatability become your responsibility. A `creates` guard can skip a command when a path exists; that proves only that the path exists, not that the full desired configuration or schema is correct. Command module.

## 4.2 Learn the precedence relationships you need

The useful question is "which definition supplies this value at this task?" Memorising the entire precedence table is less helpful than being able to trace one surprising result. This deliberately incomplete ladder shows common cases, from lower to higher priority:

```
role defaults
inventory group variables
inventory host variables
play variables
role vars
task variables
extra variables (-e)
```

It omits several forms, including registered variables and `set_fac t`. Within inventory, child-group values override parent-group values; hosts override groups. Conflicting peer groups have defined merge ordering, but relying on alphabetical accidents makes a repository hard to review. The full rules matter when several sources define the same key. Variable precedence, inventory merging.

Connection variables expose an especially plausible mistake. Ordinary CLI `-u` does not simply override every `ansible_user` variable. Variables can outrank play keywords and ordinary command-line options; `-e ansible_user=...` has the high priority of extra variables. Consequently "the command line always wins" is wrong. When debugging the SSH identity, inspect both connection variables and how the invocation supplies them. General precedence.

For the example, `status_release` in inventory overrides the role default. An extra variable can override both. Avoid a shared `group_va rs/all.yml` beside the playbook silently competing with environment-local values. Playbook-adjacent and inventory-adjacent variable loading have documented ordering; the simplest reviewable design is fewer competing definitions.

The architecture recommendation is to make environment intent explicit: connection and service choices beside each environment inventory; overridable role inputs in `defaults`; operationally fixed implementation constants used sparingly. Prefix role-owned inputs, as `status_...`, to reduce collisions. Putting every value in `roles/.../va`

`rs/main.yml` makes caller overrides harder because role vars have high precedence.

## 4.3 Grow structure in response to pressure

A personal-host play can remain a single file. A role earns its cost when a coherent responsibility has multiple files, callers, environments or maintainers. "Manage this service's configuration and activation" is a good boundary. A role per trivial task obscures ordering; one universal role containing every Linux choice makes every change harder to reason about.

For Jellybooks, separate baseline responsibilities from service-specific ones. For example, access policy and monitoring might be shared, while a particular application's configuration belongs to its service owner. Agree which role owns a shared file. Two roles rewriting the same file can each appear correct in isolation and still fight on every run. A service role should document its inputs, files, packages, ports, handler effects and verification responsibilities. Role conventions.

There are two common ways to reuse task lists. Static `import_tasks` expands a known task list during playbook processing; dynamic `include_tasks` evaluates inclusion as execution reaches it. That affects visibility in task listing and how conditions and tags apply. Prefer the simpler mechanism that matches the need, and ask for an execution trace before accepting a dynamic include tree whose order is difficult to see. Do not add an operating-system abstraction until you really support the second system and can test its differences. Imports and includes.

**Ask an agent:** "Where does the effective `status_port` come from on this host, and why can this environment override it?" Good evidence names the definitions, precedence relationship, actual type and rendered result. A weak answer points to a default while ignoring inventory, or creates another `set_fact` to mask the conflict.

**Exercise 4 — trace.** The role default is port 8080, inventory group data says 8081 and the command supplies `-e '{"status_port": 8082}'`. Which wins? If an operator supplies `-u emergency` but inventory has `ansible_user: deploy`, can they assume the SSH login changed?

# 5 Build evidence in layers

## 5.1 A preview answers a narrower question than a run

Ansible's check mode is not a Terraform plan. It asks modules to predict changes against the state they can currently observe. It does not build a complete hypothetical machine and then execute the rest of the play against it. Unsupported operations can skip; a later operation may encounter a missing file or binary that an earlier task only predicted installing. A task with `check_mode: false` can execute for real even during `--check`. Audit such tasks before using a preview as an operational safety boundary.

`--diff` shows supported before-and-after data. Useful evidence can also be sensitive evidence: configuration differences may disclose credentials. Apply it selectively and read task-level `diff` and logging policy first. Check and diff modes.

Our fresh-host nginx preview can therefore fail or be incomplete. Predicting nginx installation does not create `/usr/sbin/nginx`, the destination directory or a systemd unit. The health query is intentionally skipped. That is a limitation to explain, not a reason to add error suppression until the preview looks green.

Use the following ladder when reviewing a proposed change:

| Evidence | Establishes | Leaves open |
|---|---|---|
| Read diff and effective inputs | Intended ownership and bounded change | Runtime behaviour |
| Syntax check | Playbook can be parsed in this setup | Correct targets, values and effects |
| Host/task listing | Selection and much visible structure | All dynamic paths and effects |
| Lint | Selected rules and suspicious patterns | Functional acceptance |
| Check/diff | Supported predictions on current state | Fresh-host and activation behaviour |
| Disposable-host run | Observed behaviour in that environment | Other environments and real traffic |

| Evidence | Establishes | Leaves open |
|---|---|---|
| Repeat and drift tests | Convergence and repair for tested changes | Every possible interruption |
| Client-path acceptance | Tested end-to-end outcome | Untested paths and future conditions |

For example, an nginx configuration can parse, install and reload while serving the wrong virtual host. A matching HTTP 200 is still weak if a cached page or wrong machine returns it. The exact identity and release in our body checks are small but purposeful acceptance criteria. In a real service, add functional checks that distinguish a live process from a useful service.

## 5.2 Try the complete remote example later

On a Linux controller with Python 3.12 and the `venv` module available, from the supplied `examples/` directory:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
ansible --version
ansible-config dump --only-changed
```

Install the exact Core pin, 2.21.5, from `requirements.txt`. `ansible.cfg` enables host-key checking and sets five forks. Inspect your environment for overriding configuration. Dependency installation needs internet access; do it before travelling if you plan to use a terminal. The reading itself needs none.

Prepare **two dedicated Ubuntu 24.04 VMs**, replace the placeholder addresses, adjust the SSH account, establish trusted host keys and verify SSH plus sudo. No example creates VMs, credentials, networking or firewall rules. Then:

```
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --syntax-check
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --list-hosts --limit lab-status-01
```

```
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --list-tasks --limit lab-status-01
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --check --diff --limit lab-status-01
```

Interpret the preview limitations above. When the disposable target and inputs are correct, execute the bounded real run:

```
ansible-playbook -i inventories/lab/hosts.yml \
  site.yml --limit lab-status-01 --ask-become-pass
```

Omit `--ask-become-pass` when the lab's approved sudo setup does not require a password. Repeat the same command immediately. Expect the health assertion to run again, but no configuration change or reload when inputs and relevant state are unchanged. Apt cache policy means "unchanged" is not a promise about a run hours later.

Change `status_release` to `r2`, review the diff and run only that host again. The response must change. Then run the second lab host. A manual drift test on a disposable VM can alter a harmless response label in the configuration: the next run should restore the desired file, reload and restore the expected body. Never use a production host as a demonstration of drift repair.

For an independent observation, use the selected host's SSH address and your normal account to query its loopback endpoint:

```
ssh deploy@YOUR_LAB_ADDRESS \
  'curl -fsS http://127.0.0.1:8080/healthz'
```

This deliberately remains a host-local path. To test an external service later, query from the client's network through the real DNS, TLS, firewall, routing and load-balancer path. Merely delegating the loopback query to the controller would query the controller's loopback instead of the target's.

## 5.3 A local experiment in convergence

The supplied `local/converge.yml` uses `connection: local`, no become and no remote hosts. It writes only `local/.state/converge/` beneath the example directory. It creates a private directory, writes

`message.txt`, notifies a handler that writes `applied.txt`, flushes the handler, then reads and asserts the actual message. The meaningful core is:

```
- name: Write the desired message
  ansible.builtin.copy:
    content: "{{ lab_message }}\n"
    dest: "{{ lab_root }}/message.txt"
    mode: '0600'
  notify: Record application
```

This excerpt depends on the complete supplied playbook's directory task, variables and handler. Run the complete file, not the excerpt:

```
ansible-playbook -i localhost, local/converge.yml
ansible-playbook -i localhost, local/converge.yml
ansible-playbook -i localhost, local/converge.yml \
  -e '{"lab_message": "r2"}'
```

From an absent exercise directory, the first run changes the directory, message and handler output. The second should report `changed=0`, with no handler. The third changes the message and applied file. On an already converged exercise, `--check --diff -e '{"lab_messa ge":"r3"}'` should predict changes while leaving both existing files untouched. A check run on an entirely fresh exercise can fail because the predicted directory is not created: the same dependency issue as the fresh-host service example.

The experiment models notification and observed state; it does not model systemd, network failure or an application's reload semantics. Its assertions are useful because they examine resulting bytes instead of merely repeating the requested operation in a second task.

**Ask an agent:** "What can your tests fail to detect?" A good answer names untested target distributions, privilege boundaries, fresh-host dependencies, repetition, interruption and external request paths as applicable. "Syntax, lint and check all passed" is an evidence list, not an explanation of coverage.

**Exercise 5 — acceptance.** A change reports `changed=0` on its second run and HTTP 200 from the controller. Write three additional acceptance criteria before you would trust it on a Jellybooks service.

Make each identify a failure that those two observations could miss.

# 6 Control a fleet change

## 6.1 Concurrency is not an availability policy

Thirty to fifty machines do not call for a complicated framework by themselves. They do call for an explicit change budget. Five forks mean up to five worker processes; they do not guarantee only five services become unavailable. With no batching, a restart task can eventually restart every selected service before a later health check begins. The effect on capacity depends on the application, not just Ansible's worker count.

`serial` limits the hosts participating in a batch, and the **whole play** completes for that batch before the next batch begins. Our `serial: 1` brings one host through configuration, activation and health verification before taking another. It is conservative and slow, and still does not drain traffic or make a single-instance stateful service safe to restart.

A possible later policy, shown as an excerpt rather than a universal default, is:

```
hosts: web
serial: [1, 5]
any_errors_fatal: true
```

It processes one host, then five, then further batches of five. The list is not "one canary followed by exactly five hosts and stop". A named canary run followed by a separate reviewed fleet run can be easier to reason about than implicit ordering. Inventory order should not be assumed to be the human-written file's order. If a specific host must be first, select it explicitly. Batch execution.

Choosing a batch size requires evidence: redundancy, maximum tolerable loss of capacity, dependency topology, readiness delay and realistic failure behaviour. One host per batch is already too much if it is the only database. Five hosts per batch may be fine for fifty independent replicas and unacceptable if those five contain an entire shard's quorum. Group by failure domain and responsibility when

those distinctions matter; do not use a percentage as a substitute for understanding the service.

## 6.2 Decide what failure should stop

By default, losing one host does not necessarily stop healthy hosts or later batches. `any_errors_fatal: true` makes a failure fatal to the wider run: the current failing task completes across the batch, then execution aborts. It does not cancel already executed side effects or roll them back. With a one-host batch this gives a simple progression gate. With a five-host batch, several hosts may already have changed by the time one fails its health query.

`max_fail_percentage` is another policy mechanism: it uses a percentage threshold that must be **exceeded**, evaluated with serial batches when present. That can be appropriate when a service tolerates some host failures, but it is easier to misread than a strict stop-on-failure policy. Do not mix thresholds casually or assert a tolerance without working through concrete host counts. Aborting on errors.

A successful `rescue` can turn a failed task into a handled failure, allowing continuation under policies that would otherwise abort. This matters in rollout code: "restore the old config" does not necessarily mean "continue the next batch". If policy requires stopping after a rescue, finish with an explicit failure after recording recovery evidence. An unreachable host is a different failure class and does not enter ordinary block rescue. Blocks and rescue.

## 6.3 Turn a batch into a service-aware procedure

For a redundant web tier behind a load balancer, the intended sequence might be:

1. Prove remaining capacity and identify the host's membership.
2. Drain that host and observe connection draining or the agreed timeout.
3. Apply and validate configuration or a versioned application artifact.
4. Activate the new state and check local readiness.

5. Restore membership and check service behaviour through the real client path.
6. Proceed only when the next batch is allowed by the change policy.

Those are design obligations, not runnable load-balancer code. The API, authentication, timing and compensating actions depend on the real platform. The nginx fixture implements only configuration, activation and host-local verification. It cannot establish spare capacity or drain Jellybooks traffic.

A task with `delegate_to: localhost` runs its action on the controller, useful for a load-balancer API the target cannot reach. The loop still represents each selected inventory host; delegation does not automatically make an operation once-only or serialise concurrent writes to shared state. `run_once` with `serial` runs once **per batch**, which is a poor accidental implementation of a global schema migration. Make genuinely global operations an explicit separate phase with their own locking and evidence. Delegation.

Consider failure while drained. An `always` section that blindly re-adds every host can return an unhealthy service to traffic. An `always` section that only reports and leaves membership unchanged may require an operator to restore capacity. Neither policy is universally right. Define what health permits re-entry, what happens if the API becomes unreachable and how the operator identifies a stranded host. Recovery is part of the procedure before the first production run.

Reboots deserve the same treatment. The `reboot` module handles the expected connection interruption and waits for the machine to return, unlike a detached shell reboot command. A returned SSH session still does not prove the application has rejoined its cluster or regained readiness. Keep reboot batches and application checks tied to the availability budget. Reboot module.

**Ask an agent:** "How many hosts can be changed when the first failed health check stops this rollout, and which may still be drained?" Good evidence traces one concrete batch, task ordering, failure policy, rescue and membership state. A weak answer points only at `forks: 5` or promises that `serial` ensures zero downtime.

**Exercise 6 — predict.** A 12-host play has `serial: [1, 5]`, an

`y_errors_fatal: true`, and a successful first batch. One host in the second batch fails after all five have installed new configuration. How many hosts might already have changed? What happens to the remaining six? Would adding `run_once` to a migration make it globally once-only?

# 7 Diagnose partial completion and recover deliberately

## 7.1 Reconstruct the last proven state

A failed run is an incident with a timeline. First preserve the command, selected hosts, effective input revision, recap and relevant task results. Then ask which changes succeeded on each host. The most useful distinction is often between "desired file installed" and "process activated it". Do not let an agent edit random tasks until a rerun happens to look green.

For the nginx fixture, these are observations on the **affected target**, using its normal SSH and sudo access as required:

```
sudo nginx -t
sudo systemctl status nginx --no-pager
sudo journalctl -u nginx --since '20 minutes ago'
sudo tail -n 50 /var/log/nginx/error.log
curl -fsS http://127.0.0.1:8080/healthz
```

Check the actual configured port and file content too. These commands are investigation, not a blanket recovery script. A status command can exit nonzero when it has supplied useful evidence. `nginx -t` checks the installed config's syntax and referenced-file access; it does not establish that the running workers use it or that it produces the correct application behaviour.

| Symptom | Plausible cause | Evidence that distinguishes it |
|---|---|---|
| `UNREACHABLE` before tasks | Address, SSH, trust, user or network problem | Exact SSH failure and effective connection values |

| Symptom | Plausible cause | Evidence that distinguishes it |
|---|---|---|
| Facts fail | Python or become problem | Interpreter path, SSH query, sudo policy |
| Candidate validation fails | Bad syntax or missing referenced resource | Validation stderr and candidate's dependencies |
| Disk says `r2`, response says `r1` | Activation missed or rejected | Handler output, nginx log and live response |
| Local health passes, clients fail | Routing, TLS, firewall, DNS or balancing | Request from the real client path |
| Same template changes every run | Unstable rendering or competing writer | Diff contents and both ownership paths |

Before replacing a failed task, test a hypothesis that separates plausible causes. A missing interpreter is not corrected by changing a service timeout. A reload failure is not corrected by marking the template `changed_when: false`. A wrong target is not corrected by repeatedly running the same valid module.

## 7.2 The lost-notification trap

Imagine this task sequence on an already running host:

```
1. New configuration installed; handler notified.
2. An unrelated later task fails.
3. Pending handler is not run on that failed host.
4. Operator fixes the unrelated failure and reruns.
5. Configuration bytes already match; no new notification.
6. Old live configuration can remain active.
```

This is a consequence of task-level change detection and handler failure behaviour, not a contradiction of idempotence. The file has converged; the live service has not. Pending notifications are not a durable recovery queue carried between invocations. Our example flushes handlers before the health query to make the intended activation point explicit. That narrows the failure window; it cannot remove every

interruption between file installation and activation. Handlers after failure.

`force_handlers` can make notified handlers run despite some later failures. It cannot reach an unreachable host and is not a general correctness guarantee. It also needs a policy judgement: after certain failures, applying a pending configuration may be exactly what you do not want.

The supplied `local/handler-gap.yml` reproduces the lost notification with files. `disk.txt` models installed config and `applied.txt` models activation. On the first run a copy changes disk, then a deliberate failure prevents the handler. On the second, the failure is disabled but the unchanged copy sends no new notification. The applied file remains absent:

```
ansible-playbook -i localhost, local/handler-gap.yml
ansible-playbook -i localhost, local/handler-gap.yml \
  -e '{"lab_fail": false}'
```

The first command is expected to exit nonzero. Inspect `local/.state/handler-gap/`: `disk.txt` should contain `r2`, while `applied.txt` does not yet exist. The second command can report `changed=0` despite that incomplete outcome. Then explicitly apply the already inspected state:

```
ansible-playbook -i localhost, local/handler-gap.yml \
  -e '{"lab_fail": false, "lab_apply": true}'
```

That recovery copies the desired bytes to `applied.txt`. It demonstrates a purposeful missing transition, not a recommendation to write a copy command as a service reload. The lab is stateful: later reruns see those files. For an independent repeat, choose a new private absolute exercise directory using `-e '{"lab_root":"/tmp/jeff-handler-gap-2"}'` on **all three commands**. Do not accidentally reuse a directory with an existing applied file.

## 7.3 Recovery is a new, bounded change

If the real host's installed config is the intended one, validate it and perform the missing reload under the agreed change policy; then check the served result. If the installed config is wrong, restore the intended

known-good configuration, validate, activate and verify that instead. Preserve the failed candidate and logs when they matter for diagnosis. Do not reload simply because a file exists.

nginx adds its own behaviour: when a reload cannot apply its new configuration, it can keep serving with the old configuration. A successful signal or a running master process is weaker evidence than the expected response plus clean logs. nginx configuration reload.

A Git revert restores repository content. It does not reverse a package upgrade, database migration, sent email, deleted data or remote API operation. A template backup restores a file, not the rest of a host. Rolling back a service may require compatible configuration, executable, data format and dependencies. For a stateful system, agree backup and restore semantics with the service owner rather than promising a universal "rollback playbook".

Be cautious with `--start-at-task` and `--tags` during recovery. They can omit setup, variable registration, validation or the task that would notify a handler. A targeted full rerun is often easier to reason about **when** repeatability is established and the missing activation is handled explicitly. When it is not, write a bounded recovery procedure from the observed intermediate state.

**Ask an agent:** "The file is current but the response is stale. Show the smallest evidence-backed recovery, including why a plain rerun might fail." A good answer inspects the installed file, live response and activation evidence, then proposes a validated transition and acceptance check. "Delete the file so Ansible notices a change" is a warning sign: it manufactures state loss to trigger an action whose reason should be explicit.

**Exercise 7 — diagnose.** After a failed rollout, the next run reports `changed=0`, nginx is active, and the exact health check still returns `r1` instead of `r2`. Give two explanations and the observations that distinguish them. What must a recovery prove before moving to the next host?

# 8 Keep operations understandable

## 8.1 Treat secrets as a data path

Ansible Vault encrypts files or values **at rest**. Once unlocked, a secret is available to the controller and to tasks that use it. Encryption of a repository file does not prevent decrypted values appearing in a target file, debug output, module arguments, diff, process output or a retained job log. Decide who can obtain the decryption credential, where plaintext must exist and how it is protected at each step. Vault boundaries.

For a real application, a secret-bearing template task might look like this **excerpt**, with its source template, secret variables and application account provided by that application's role:

```
- name: Install the application's private environment file
  ansible.builtin.template:
    src: application.env.j2
    dest: /etc/application/private.env
    owner: root
    group: application
    mode: '0640'
  no_log: true
  diff: false
```

This example intentionally does not invent a production secret or an account. `no_log` reduces disclosure through task logging; `diff: false` disables a potentially revealing diff. Neither rescues a later debug task that prints the same secret or an application that logs its environment. File permissions must match the process's legitimate access, not a blanket `0644`. Keep Vault passwords or external-secret credentials outside the repository, restrict log access and avoid echoing inventory containing decrypted values. Logging.

Vault files are one reasonable choice for a small fleet. An existing secret manager can improve rotation and central access policy, at the cost of controller credentials, plugin dependencies and runtime availability. Choose based on your operational needs. Do not introduce a new secret system solely because an agent has a convenient example using it. Review the credential's scope and the plaintext path before

debating which storage mechanism is fashionable.

Privilege also belongs in the data path. A controller that can deploy root-owned configuration to fifty hosts is a powerful operational system. Limit who can change and execute its repository, where its SSH keys live and what its automation account can do. The example's root-level become is convenient for a disposable lab; a production policy must be deliberate. Be cautious about delegated tasks in privileged plays: inherited become settings can affect the delegated machine unless overridden appropriately.

## 8.2 Pin the executable environment, then plan upgrades

`ansible-core==2.21.5` pins the tutorial's engine, not every dependency forever. The example uses no external collection. Once a repository uses community or vendor modules, record collection requirements and tested versions as well. A collection's own version is separate from Core's. This **illustrative** file shows the shape, not a recommendation of a collection or version for Jellybooks:

```
collections:
  - name: your_namespace.your_collection
    version: '1.2.3'
```

Replace placeholders with an actual researched dependency before installation. For a real collection requirements file, `ansible-galaxy collection install -r requirements.yml` installs its collections. Pinned external roles use their own requirements entries and installation workflow. An execution-environment image can package the controller's runtime and dependencies for consistent CI and operator use; it is a controller environment, not an agent installed on each managed host. Collection installation, execution environments.

A pin gives you a reproducible starting point, not permission to ignore fixes. Review maintenance status and porting guides during upgrades. Core 2.19 introduced substantial templating changes, so an inherited repository written for older versions deserves actual upgrade testing, especially around types and conditionals. Test the existing behaviour under its known environment first; then change the engine

or dependencies as a separately reviewable step.

Do not let two operators or CI jobs run mutating plays against the same targets without an agreed coordination mechanism. Ordinary Ansible invocations do not provide a fleet-wide lock shared across controllers. A CI environment lock or other shared change lock can be sufficient at your scale. Record the revision, inventory snapshot, invocation and outcome so you can reconstruct a change even when a dynamic inventory later changes membership.

## 8.3 Read an inherited repository before refactoring it

Start with its documented entry point, actual invocation, environment inventory, configuration file and dependency pins. Read one host's execution path through roles, includes and handlers. Trace a service input to its rendered file and acceptance check. Look for dynamic inventory plugins and variable loading beside both the playbook and inventory. A folder tree alone cannot establish the execution path.

Next establish a baseline on disposable targets or the existing safe test setup. Make the requested bounded change without simultaneously reorganising the whole repository. A working unconventional naming scheme may be less urgent than an unverified restart handler. Refactoring can come later, with unchanged behaviour as an explicit acceptance criterion.

Tags select work, not complete dependency closures. A `--tags conf ig` run can omit package installation or another task whose registered data a template needs. A tag name is not evidence that an operation is safe in isolation. Static imports and dynamic includes also expose tags differently. Review what the precise invocation will actually run and document which partial entry points are supported. Tags.

Plausible generated changes deserve concrete scrutiny:

| Proposal | Behavioural problem | Better review question |
|---|---|---|
| Shell installs every package and appends config | Uncontrolled repetition and weak change reporting | Which modules can express the state and repair drift? |
| `changed_when: false` on a reload | Hides mutation rather than avoiding it | Why should this reload run, and what proves it did? |
| `ignore_errors: true` around readiness | Lets an unusable service pass the progression gate | Which failure is acceptable, and where is the gate enforced? |
| `state: latest` across all hosts | Couples the run to moving repository content | Is this an explicit patching change with a version and disruption policy? |
| One root shell command with pipes | Hides intermediate failure and quoting assumptions | Can each consequential operation be observed and repeated safely? |
| A role for every two-line task | Adds indirection without a responsibility boundary | What shared contract or ownership justifies it? |

These are review heuristics, not bans. A shell command can be the right tool for a missing module, and accepting an expected error can be deliberate. Require an explanation grounded in the external programme's behaviour, rather than a rule applied mechanically. In particular, never improve a recap by redefining failure until it stops expressing the acceptance condition you care about.

**Ask an agent:** "Make only the requested service change, and show its plaintext secret path, dependency changes and supported invocation." Good evidence is a small diff tied to existing conventions and observed tests. A weak answer adds a secret backend, role framework and dependency upgrade without showing why the requested change requires them.

**Exercise 8 — security review.** A proposal encrypts `secrets.ym`

`l` with Vault, uses its contents in a template with mode `'0644'`, and enables `--diff` in CI. The author says the values are safe because Git contains ciphertext. Trace two plaintext exposure paths and propose controls matched to those paths.

# 9 Share ownership across tools

This optional chapter assumes the execution, inventory and failure models from chapters 1–7. Its purpose is to help you specify a handover between tools; it is not a Terraform or container orchestration tutorial.

## 9.1 Establish a single writer for each resource

A useful initial boundary for your roadmap is: Terraform owns the lifecycle of VMs, networks, addresses and related provider resources; Ansible owns agreed packages, host configuration and services inside those machines. This is an architectural recommendation, not a restriction on what either tool can do. Ansible has cloud modules; Terraform has provisioners. Overlapping capabilities do not make overlapping ownership desirable.

Terraform maintains resource state to relate configuration to provider-managed objects and plan changes. Ordinary Ansible host configuration discovers current state during each run; it does not rely on an equivalent Terraform resource-state file tracking every managed package and file. Neither tool thereby guarantees a multi-system transaction. A VM created successfully followed by a failed host configuration run is a real intermediate outcome to record and recover. Terraform state purpose.

HashiCorp recommends purpose-built configuration tools rather than relying on Terraform provisioners for routine configuration management. Provisioners have behaviour Terraform cannot model as predictably as provider resources. A separately visible Ansible phase makes its targets, failure and recovery easier to inspect. Avoid hiding a fleet configuration run in a `remote-exec` script simply to obtain a one-command workflow. Terraform provisioners.

Write the handover contract before choosing the integration script:

| Contract element | Example question |
|---|---|
| Identity | What stable host identity maps Terraform output to inventory? |
| Reachability | Which address and SSH path does the controller use? |
| Bootstrap | Does the image provide Python, account, trust and sudo? |
| Readiness | What proves SSH and package management are ready? |
| Membership | Which environment and service groups should contain the host? |
| Removal | Who drains a host before Terraform destroys it? |
| Recovery | How is "created but not configured" represented and retried? |

Feed machine-readable Terraform outputs or a maintained provider inventory plugin into the inventory process. Validate the resulting membership rather than maintaining a second handwritten list of changing addresses. Decide whether a run uses a frozen snapshot or discovers membership live. Dynamic inventory is useful automation; its query, filters, credentials and refresh behaviour become part of target selection and need the same review as a static file.

For example, a role tag supplied by the infrastructure layer might determine the `web` group. A typo in that tag can make a perfectly valid play target zero hosts or the wrong service. The right acceptance test is the expected identity set and bootstrap contract, followed by host configuration and service acceptance.

## 9.2 Containers move some of the boundary

For Dockerised services, place application dependencies and immutable executable content in the image where practical. Ansible can still own the host's Docker installation, storage, access, monitoring and the agreed service-launch configuration. Pin application artifacts by an immutable identity where the release policy requires it; a moving tag

makes “same YAML” weaker evidence of “same executable”. Decide who owns each layer rather than installing the same application dependencies both in a host role and an image recipe.

For a non-Docker service, an Ansible role can own a systemd unit and configuration while consuming an artifact produced by CI. Packaging the application as a versioned OS package is one option; downloading and verifying a versioned artifact is another. Installing from an unpinned source checkout and running `pip install -U` on every host makes reconstruction and rollback much harder. Use Python virtual environments deliberately, and separate application dependency versions from the Python needed for Ansible’s own target modules.

If Ansible writes a systemd unit, it must make systemd reread unit definitions and then perform the appropriate service transition. If it writes an application’s configuration, the application’s reload or restart semantics apply. Both may be needed; one is not a substitute for the other. Decide which files require a restart, which support reload and what health establishes after each.

A future pod-based platform may move service lifecycle and health management to a scheduler. If that platform is Kubernetes, for example, the controller reconciliation model differs from a periodic Ansible host run. Avoid having two systems independently restart, replace or rewrite the same workload. Your specific future platform remains unspecified, so the next useful decision is a resource ownership map, not a Kubernetes role collection.

**Ask an agent:** “Draw the ownership boundary and explain recovery if Terraform succeeds but Ansible fails.” A good answer describes durable infrastructure, incomplete configuration, a stable inventory contract and a bounded retry or removal procedure. A weak answer treats VM creation and service readiness as one undifferentiated success or proposes two tools managing the same configuration.

**Exercise 9 — choose a boundary.** Terraform creates a VM; cloud-init installs and rewrites nginx configuration on each boot; an Ansible role rewrites the same file; the application deployment script also restarts nginx. Describe one simpler ownership contract. What must happen before Terraform may remove a VM that still serves traffic?

# 10 Worked answers

Attempt the exercises first. These answers show reasoning rather than a single required phrasing. Your service topology may justify a different policy, but it must still explain the actual execution semantics.

## 10.1 1. Earlier effects survive

The installed configuration and service start remain. A failed health check does not undo them. Inspect the precise selected host, installed configuration, validation result, process state, activation evidence and actual response. Then determine whether rerunning repeats only safe operations and whether any activation step must be made explicit. If the health failure reflects an application regression, a blind retry may only reproduce it.

The distinction you should now be able to judge is whether "retry" describes a supported transition from known state, rather than hope that Ansible is atomic.

## 10.2 2. A limit must resolve to the intended inventory identity

Check `--list-hosts` with the exact inventory and invocation. The IP-valued limit may match no alias even though the alias's `ansible_host` contains that address. Use `lab-status-01` when that is the intended identity, and prove its effective connection address. Inline inventory may drop the account, interpreter, groups and service variables attached to the real inventory. A zero-target or differently configured run is not a successful implementation of the request.

You can now distinguish a target-selection fix from a shortcut that silently changes the execution context.

## 10.3 3. Report and control the right transition

An ordinary `state: restarted` task restarts on every invocation, even when the configuration is unchanged. If all that is needed is to activate changed nginx configuration, a notified handler avoids unnecessary disruption. A restart can still be appropriate for changes

that require one; prove the service's supported reload behaviour and test the resulting live state.

`daemon_reload` addresses systemd's unit definitions, not nginx's own config. A running unit and a successful service-management command do not establish the served release. Inspect documentation, logs and the actual acceptance response.

You can now judge whether a service action is both necessary and sufficient.

## 10.4 4. Extra variables win here; ordinary CLI flags need care

The JSON extra variable supplies integer 8082, overriding inventory and the role default. A plain `-u emergency` does not necessarily override inventory's `ansible_user: deploy`; connection variables outrank ordinary options. Trace the actual definitions or explicitly use the intended high-priority variable under your access policy. Do not fix an unexplained precedence conflict by scattering new assignments throughout the play.

You can now judge a value by provenance and type, not just by the nearest YAML.

## 10.5 5. Convergence and a 200 response leave important gaps

Three strong additional criteria might be:

1. The exact intended host set is selected, with expected identities and no duplicate physical target. This catches success on the wrong machine.
2. Each affected host serves the expected release or functional result through the real client path after activation. This catches a stale process, cached controller result, broken TLS or routing despite a 200 somewhere else.
3. A disposable fresh host and a deliberately drifted host both reach the intended state, with an unchanged repeat run and no unnecessary activation. This catches missing bootstrap assumptions and hidden competing writers.

For a stateful service, add an appropriate data correctness and recovery criterion. For a secret change, add permissions and log-disclosure checks. A repeat run proves only the state and paths you actually exercised.

## 10.6 6. A batch can change several hosts before its gate fails

Up to six hosts may have changed: the first host, then all five in the second batch. The failed task completes within the current batch; the fatal-error policy prevents progression to the remaining six hosts. Already changed hosts stay changed. Membership and activation states require inspection.

`run_once` is once per serial batch. It does not make a schema migration a single operation across the entire rollout, nor does it establish idempotence or an external lock. A distinct migration phase with a service-specific contract is clearer.

You can now judge the maximum scope of an intermediate outcome rather than assuming a canary policy limits all failures to one host.

## 10.7 7. Separate missed activation from rejected activation

One explanation is the lost notification: disk changed during the failed run, the handler never ran, and the rerun saw identical bytes. Another is that a reload was attempted but nginx could not adopt the new configuration, or that the request reaches a different process, host or endpoint than assumed. A generic `systemctl is-active` cannot distinguish these.

Check handler output and timing, validate the actual file, read nginx's error log, identify the request endpoint and compare its exact response with the installed release. Recovery must establish the intended valid configuration is installed and active on the intended host, then prove the relevant health criterion. Only then should the rollout gate reopen. If traffic was drained, verify the membership transition as well.

## 10.8 8. Ciphertext in Git does not protect every plaintext copy

The target file is readable to every local user under `'0644'`. Set ownership and permissions appropriate to the actual reader, such as a dedicated application group with `'0640'`, or root-only `'0600'` when appropriate. Check directory and backup permissions as well.

A CI diff can print decrypted file content into retained logs. Disable diffs for the secret-bearing task, apply `no_log` where secret-bearing results could be logged and remove subsequent debug output of the value. Protect decryption credentials and job-log access. Also examine the application itself: neither Vault nor task logging policy prevents it from logging a secret after startup.

You can now judge secret protection along a complete data path rather than from the repository format alone.

## 10.9 9. Give the file and lifecycle one owner each

A reasonable contract is: Terraform owns VM/network lifecycle and exports identity and connection data; cloud-init only establishes the bootstrap account, Python and access contract; Ansible owns nginx's host configuration and activation; the application pipeline requests an agreed deployment operation instead of independently rewriting or restarting nginx. Another architecture could delegate application-facing nginx entirely to a container platform, but it must still remove overlapping writers.

Before removal, drain the machine through the real traffic-management mechanism, verify the service no longer depends on it, handle state and persistent data, then allow infrastructure destruction. Establish what happens if drain succeeds but destroy fails, or destroy is requested while configuration is incomplete. You can now judge a handover by identity, ownership and failure outcomes.

# 11 A field reference for later

Use this after the flight when an agent proposes a change. Each question points to behaviour you have already traced; it is not a substitute for understanding the affected service.

| Question | Evidence worth asking for |
|---|---|
| What will be touched? | Exact inventory, revision, host list, connection context and owned resources |
| Where does this value come from? | Definitions, precedence, type and rendered result on one host |
| Why does this task report changed? | Module semantics or justified `changed_when`, plus a repeat run |
| How does new disk state become live? | Validation, notification, activation point and response check |
| How far can failure spread? | Batch membership, policy and concrete partial-state trace |
| What happens on retry? | Repeatability of each consequential effect, including missed handlers |
| How is the secret protected? | Plaintext locations, permissions, logging and decryption access |
| Who owns this resource? | One writer per file or lifecycle responsibility across tools |
| What remains unproved? | Named coverage gaps and the evidence needed before real adoption |

A useful change request is specific about outcomes. For example:

> Add the agreed service configuration to the two staging hosts selected by the staging inventory. Preserve existing unrelated services. Validate the candidate before replacement, activate only when required, and verify the expected release through the agreed request path. Show fresh-host, repeat-run and partial-failure evidence. Stop before the next batch if readiness fails, and explain recovery for the states already changed.

That is more useful than asking an agent to “make an idempotent Ansible role”. It states what correctness means while leaving mechanical implementation to the agent. Adapt it to actual ownership and availability policy rather than copying it unchanged into every task.

# 12 Sources, artifacts and verification

Research date: **20 October 2026**. The executable target is **Core 2.21.5**. Official Ansible community documentation URLs under `latest` can change after this date; the version pin and recorded observations identify what this tutorial actually targeted. Module help installed with the pinned engine, for example `ansible-doc ansible.builtin.template`, is another useful local reference.

Primary sources are linked beside the claims they support. The main reading routes for later reference are:

- Ansible concepts, inventory and variable precedence.
- Handlers, error handling, strategies and check mode.
- Roles, Vault and the Core porting guide.
- nginx controls and Terraform provisioner guidance.

The source package keeps one tutorial directory. It contains `brief.md` (learner context and assumptions), `outline.md` (capabilities mapped to those goals), `tutorial.md`, `ansible.pdf`, `Makefile`, `pdf-header.tex`, `README.md`, `verification.md` and the complete `examples/` tree. Everything needed to rebuild the PDF is in that directory apart from the documented toolchain.

Verification separates executed behaviour from documented behaviour. The local convergence and handler-gap exercises were executed with Core 2.21.5 and Python 3.12. Their repeat, preview and recovery outcomes matched the explanations. The remote playbook was syntax-checked and its inventory selection inspected. The actual nginx templates were rendered with Ansible; Ubuntu-container checks validated the complete configuration, confirmed the exact `r1` response, reloaded to `r2`, and rejected an invalid candidate. The actual health-query expression also accepted the expected line and rejected both a wrong release and a missing final newline in a local HTTP fixture.

**Limits:** the complete SSH/sudo/systemd play was not executed against full Ubuntu VMs. Container checks do not establish systemd behaviour, boot enablement, SSH trust, target sudo policy, live fleet availability or external traffic paths. The unit-management claims were checked against official module documentation. Before real adoption, use the disposable-VM sequence in chapter 5 and adapt its acceptance criteria to your service. No live machines were used for verification.